

OpenSSL 연동 구현 설명서

Technical Documentation

김주현

June 20, 2026

Contents

1 개요 (Overview)	3
1.1 프로젝트 목적	3
1.2 주요 구성	3
1.3 지원 알고리즘 범주	3
2 설치 및 빌드 (Build & Install)	3
2.1 현재 확인된 빌드 특성	3
2.2 권장 빌드 절차	3
2.3 알고리즘 오브젝트 빌드 예시	4
2.4 산출물 확인	4
2.5 테스트 실행 예시	4
3 라이브러리 구조와 핵심 모듈 (Architecture)	4
3.1 디렉터리 구조	4
3.2 레이어 관점	4
3.3 KEM/SIG 디스패치 모델	5
3.4 정적 라이브러리 분리	5
4 공개 API 사용 가이드 (Public API)	5
4.1 핵심 헤더	5
4.2 라이프사이클 API	5
4.3 KEM API 흐름	5
4.4 SIG API 흐름	6
4.5 메모리 API 권장 패턴	6
4.6 알고리즘 선택 시 주의	6
5 예제 코드와 실행 결과 설명 (Examples)	6
5.1 KEM 기본 사용 예제	6
5.2 실행 결과 해석	7
5.3 테스트 코드 기반 참고	7
6 성능/보안 고려사항 (Performance & Security)	7
6.1 성능 관점	7
6.2 보안 관점	7
6.3 운영 권장사항	7
7 한계점 (Limitations)	7
7.1 문서 범위	7

8	개발 규칙 (Development Rules)	8
8.1	코딩/리뷰 규칙	8
8.2	테스트 규칙	8
8.3	문서 유지보수 규칙	8
8.4	참고 링크	8

1. 개요 (Overview)

1.1. 프로젝트 목적

본 문서는 liboqs-analysis의 KEM(Key Encapsulation Mechanism)과 전자서명(Signature, SIG) 구현 구조, 빌드 산출물, KAT 테스트 흐름을 정리한다. 현재 프로젝트는 KEM/SIG 알고리즘을 공통 API로 선택하고 실행하는 흐름을 중심으로 구성되어 있다.

1.2. 주요 구성

- 공개 인터페이스: include/oqs/*.h
- 통합/공통 헤더: include/oqs/oqs.h, include/oqs/common.h
- KEM/SIG 헤더: include/oqs/kem.h, include/oqs/sig.h
- 핵심 구현: src/common, src/kem, src/sig
- 테스트: tests/kat_kem.c, tests/kat_sig.c, tests/test_helpers.*
- 빌드 산출물: build/lib/liboqs.a, build/lib/liboqs-internal.a

1.3. 지원 알고리즘 범주

범주	예시 알고리즘	관련 경로
KEM	HQC-128/192/256, ML-KEM-512/768/1024	include/oqs/kem.h, src/kem/*
SIG	ML-DSA, SLH-DSA 계열	include/oqs/sig.h, src/sig/*
Common Crypto	AES, SHA2, SHA3, RNG	include/oqs/common.h, include/oqs/rand.h, src/common/*

2. 설치 및 빌드 (Build & Install)

2.1. 현재 확인된 빌드 특성

include/oqs/oqsconfig.h 기준:

- 버전: OQS_VERSION_TEXT = 0.15.0
- 플랫폼: x86_64 Linux, 배포 빌드 플래그 활성화
- OpenSSL 연동: OQS_USE_OPENSSL=1
- 스레드: OQS_USE_PTHREADS=1

2.2. 권장 빌드 절차

프로젝트 루트에서 필요한 테스트 대상을 빌드한다. 예를 들어 KEM 테스트 바이너리는 다음 명령으로 생성한다.

Listing 1: 테스트 바이너리 빌드 예시

```
1 make kat_kem
```

SIG 테스트 바이너리가 필요하면 make kat_sig를 실행한다.

2.3. 알고리즘 오브젝트 빌드 예시

각 알고리즘 구현 파일은 다음과 같이 컴파일러를 직접 호출해 오브젝트 파일로 빌드할 수 있다.

Listing 2: SLH-DSA 구현 파일 컴파일 예시

```
1 /usr/bin/cc -I./include \  
2 -std=gnu11 -fPIC -fvisibility=hidden -Wa,--noexecstack -O3 -fomit-  
   frame-pointer -fdata-sections -ffunction-sections -Wl,--gc-  
   sections -DMLD_CONFIG_PARAMETER_SET=44 -DMLD_CONFIG_FILE="../../  
   integration/liboqs/config_c.h" \  
3 -MMD -MP -o build/sig/slh_dsa/wrappers/pure/slh_dsa_pure_shake_128f  
   .o \  
4 -c ./src/sig/slh_dsa/wrappers/pure/slh_dsa_pure_shake_128f.c
```

2.4. 산출물 확인

Listing 3: 주요 정적 라이브러리 산출물

```
1 build/lib/liboqs.a  
2 build/lib/liboqs-internal.a
```

2.5. 테스트 실행 예시

Listing 4: KAT 기반 KEM 테스트 예시

```
1 ./build/tests/kat_kem ml-kem-768 > /tmp/kat_mlkem768.out  
2 echo $?
```

3. 라이브러리 구조와 핵심 모듈 (Architecture)

3.1. 디렉터리 구조

Listing 5: 핵심 디렉터리 개요

```
1 include/oqs/      # public API headers  
2 src/common/      # common crypto utilities (AES/SHA/RNG/memory)  
3 src/kem/         # KEM interface and algorithm implementations  
4 src/sig/         # signature interface and algorithm implementations  
5 tests/          # KAT and helper-based validation code  
6 build/lib/       # static library outputs
```

3.2. 레이어 관점

- API Layer: include/oqs/*.h
- Dispatch Layer: src/kem/kem.c, src/sig/sig.c (알고리즘 식별자 매핑, 객체 생성)
- KEM Algorithm Layer: src/kem/hqc/*, src/kem/ml_kem/*
- SIG Algorithm Layer: src/sig/ml_dsa/*, src/sig/slh_dsa/*
- Utility Layer: src/common/* (메모리/난수/해시/대칭키)

3.3. KEM/SIG 디스패치 모델

OQS_KEM_new(method_name)와 OQS_SIG_new(method_name)는 문자열 식별자를 기반으로 알고리즘 인스턴스를 생성한다.

KEM 경로에서는 생성된 OQS_KEM 구조체의 길이 필드와 함수 포인터를 통해 keypair, encaps, decaps를 공통 방식으로 호출한다. SIG 경로에서는 생성된 OQS_SIG 구조체를 통해 keypair, sign, verify를 공통 방식으로 호출한다.

3.4. 정적 라이브러리 분리

라이브러리	역할	관련 모듈
liboqs.a	알고리즘 및 공통 구현 본체	KEM/SIG, AES, SHA, RNG 등
liboqs-internal.a	내부 링크 구조 유지를 위한 보조 아카이브	현재 Makefile에서는 빈 아카이브로 생성

4. 공개 API 사용 가이드 (Public API)

4.1. 핵심 헤더

- <oqs/oqs.h>: 통합 진입 헤더
- <oqs/common.h>: 상태 코드, 초기화, 메모리, CPU 확장 체크
- <oqs/rand.h>: RNG 선택 및 바이트 생성
- <oqs/kem.h>: KEM 객체/함수
- <oqs/sig.h>: 전자서명 객체/함수

4.2. 라이프사이클 API

함수	설명
OQS_init()	라이브러리 전역 초기화(OpenSSL/CPU feature 준비 포함)
OQS_destroy()	전역 자원 정리
OQS_thread_stop()	멀티스레드 환경에서 스레드 종료 전 정리
OQS_version()	라이브러리 버전 문자열 반환

4.3. KEM API 흐름

1. OQS_KEM_alg_identifier(i) 또는 문자열 상수로 알고리즘 선택
2. OQS_KEM_new(name)로 객체 생성
3. 길이 필드(length_public_key 등) 기반 버퍼 할당
4. OQS_KEM_keypair, OQS_KEM_encaps, OQS_KEM_decaps 호출
5. OQS_KEM_free() 및 민감정보 secure free

4.4. SIG API 흐름

1. OQS_SIG_alg_identifier(i) 또는 문자열 상수로 알고리즘 선택
2. OQS_SIG_new(name)로 객체 생성
3. 길이 필드(length_public_key, length_signature 등) 기반 버퍼 할당
4. OQS_SIG_keypair, OQS_SIG_sign, OQS_SIG_verify 호출
5. OQS_SIG_free() 및 민감정보 secure free

4.5. 메모리 API 권장 패턴

- 민감정보(비밀키/공유비밀): OQS_MEM_secure_free
- 비민감정보(공개키/암호문): OQS_MEM_insecure_free
- 비교는 memcmp 대신 OQS_MEM_secure_bcmp 우선 검토

4.6. 알고리즘 선택 시 주의

OQS_KEM_alg_count(), OQS_SIG_alg_count()와 각 alg_identifier()로 열거되는 목록이 실행 시점의 enabled 상태와 동일하지 않을 수 있으므로, 실사용 시 OQS_KEM_alg_is_enabled() 또는 OQS_SIG_alg_is_enabled() 체크를 병행한다.

5. 예제 코드와 실행 결과 설명 (Examples)

5.1. KEM 기본 사용 예제

Listing 6: ML-KEM-768 encaps/decaps 최소 예제

```
1 #include <oqs/oqs.h>
2 #include <oqs/kem.h>
3 #include <oqs/common.h>
4
5 int main(void) {
6     OQS_init();
7
8     OQS_KEM *kem = OQS_KEM_new(OQS_KEM_alg_ml_kem_768);
9     if (kem == NULL) return 1;
10
11     uint8_t *pk = OQS_MEM_malloc(kem->length_public_key);
12     uint8_t *sk = OQS_MEM_malloc(kem->length_secret_key);
13     uint8_t *ct = OQS_MEM_malloc(kem->length_ciphertext);
14     uint8_t *ss_e = OQS_MEM_malloc(kem->length_shared_secret);
15     uint8_t *ss_d = OQS_MEM_malloc(kem->length_shared_secret);
16
17     if (OQS_KEM_keypair(kem, pk, sk) != OQS_SUCCESS) return 2;
18     if (OQS_KEM_encaps(kem, ct, ss_e, pk) != OQS_SUCCESS) return 3;
19     if (OQS_KEM_decaps(kem, ss_d, ct, sk) != OQS_SUCCESS) return 4;
20
21     int ok = (OQS_MEM_secure_bcmp(ss_e, ss_d, kem->length_shared_secret) == 0);
22
23     OQS_MEM_secure_free(sk, kem->length_secret_key);
24     OQS_MEM_secure_free(ss_e, kem->length_shared_secret);
25     OQS_MEM_secure_free(ss_d, kem->length_shared_secret);
```

```

26     OQS_MEM_insecure_free(pk);
27     OQS_MEM_insecure_free(ct);
28     OQS_KEM_free(kem);
29     OQS_destroy();
30
31     return ok ? 0 : 5;
32 }

```

5.2. 실행 결과 해석

정상 흐름에서는 `keypair -> encaps -> decaps`가 모두 `OQS_SUCCESS`를 반환하고, `ss_e`와 `ss_d`가 동일해야 한다.

5.3. 테스트 코드 기반 참고

`tests/kat_kem.c`는 NIST KAT 스타일 출력 형식(`count`, `seed`, `pk`, `sk`, `ct`, `ss`)을 생성하며, 다중 카운트 검증은 `-all` 플래그로 수행한다.

6. 성능/보안 고려사항 (Performance & Security)

6.1. 성능 관점

- 빌드 플래그: `-O3`, `-fdata-sections`, `-ffunction-sections`, `-gc-sections`
- 아키텍처 최적화 분기: `oqsconfig.h`의 `x86_64` 관련 `define`, AVX 계열 조건부 사용
- 알고리즘별 trade-off: ML-KEM/HQC 파라미터 세트별 키/암호문 크기 및 연산비용 상이

6.2. 보안 관점

- 민감 버퍼는 `OQS_MEM_cleanse`, `OQS_MEM_secure_free` 경로 사용
- 공유비밀 비교는 타이밍 누설 완화를 위해 `OQS_MEM_secure_bcmp` 사용
- RNG 소스 변경 시(`OQS_randombytes_switch_algorithm`) 테스트/검증 절차 필수
- 오류 처리 시 조기 반환보다 정리(`cleanup`) 블록 일원화 권장

6.3. 운영 권장사항

1. 알고리즘 `enable` 여부를 시작 시점에 검사해 `fallback` 정책을 정의한다.
2. OpenSSL/플랫폼 업데이트 시 회귀 테스트(KAT + 상호운용성)를 수행한다.
3. CI에 최소 1개 KEM과 1개 SIG smoke test를 배치한다.

7. 한계점 (Limitations)

7.1. 문서 범위

본 문서는 현재 프로젝트의 빌드/테스트 흐름과 공개 헤더 사용법을 설명한다. 외부 종속성의 세부 ABI나 배포 환경별 패키징 차이는 별도 검증 대상이다.

8. 개발 규칙 (Development Rules)

8.1. 코딩/리뷰 규칙

1. 공개 API 변경 시 `include/oqs/*.h`와 구현체를 동시에 갱신한다.
2. 알고리즘 추가/삭제 시 식별자 목록, count, enable 분기를 일관되게 수정한다.
3. 민감 데이터 메모리 해제는 secure API를 우선 적용한다.
4. 실패 경로는 단일 cleanup 라벨로 정리해 누수/중복 해제를 방지한다.

8.2. 테스트 규칙

- KEM: `keypair/encaps/decaps` 왕복 성공 + 공유비밀 일치 필수
- SIG: 서명/검증 정상 케이스 + 변조 입력(bitflip) 실패 케이스 필수
- 알고리즘 enable/disable 설정별 smoke test 수행

8.3. 문서 유지보수 규칙

- 새 알고리즘 추가 시 본 문서 3장(구조), 4장(API), 5장(예제) 동시 갱신
- 빌드 옵션 변경 시 2장 업데이트
- 문서 범위가 바뀌면 7장을 함께 갱신

8.4. 참고 링크

- GitHub 저장소: <https://github.com/wngus7450/liboqs-analysis>
- Public headers: <https://github.com/wngus7450/liboqs-analysis/tree/main/include/oqs>
- Test entry points: <https://github.com/wngus7450/liboqs-analysis/tree/main/tests>